CH

Automated Root-Cause Analyst

Real-time anomaly detection and drilldown for ad-tech metrics — built entirely on ClickHouse.

CLICK-A-THON 2026 · INMOBI PROBLEM · TEAM HOPE

THE PROBLEM

Ad platforms generate millions of events per hour. Anomalies hide in the noise.

A fill-rate crash in one country, an eCPM collapse on a single OS version, a revenue dip across one ad format — these cost real money but stay invisible at the platform aggregate until it's too late.

Delayed detection

Manual dashboards catch issues hours or days later. By then the revenue impact is already locked in.

Hidden in segments

A 30% fill-rate crash in Indonesia is invisible when the global average only dips 2%. Segment-blind monitors miss it entirely.

Root cause unknown

Even after spotting an anomaly, teams spend hours slicing data to find *which* segment, device, or region is responsible.

OUR SOLUTION

Detect, drill down, and explain — in seconds.

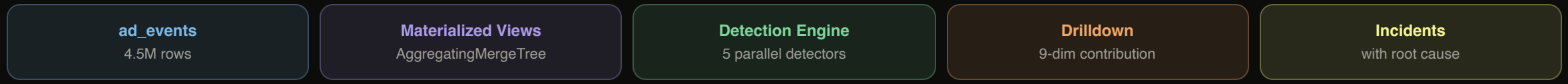
An end-to-end anomaly detection pipeline that runs **entirely inside ClickHouse** — no external stats libraries, no Python, no Spark. All math (median, IQR, CUSUM, linear regression) executes as native SQL.

5
Parallel detectors

9
Dimension axes scanned

~3s
Full detection cycle

4.5M
Events analyzed



CLICKHOUSE ARCHITECTURE

Materialized views as the computational backbone

Every insert into **ad_events** triggers materialized views that maintain pre-aggregated rollups using **AggregatingMergeTree** with `-State` / `-Merge` functions.

daily_global_agg

Platform-level daily rollup. ~40 rows total. Queried by z-score and CUSUM detectors for baseline computation. **~5ms query time** vs ~500ms on raw events.

hourly_global_agg

Hourly rollup for sub-day detection windows (1h, 6h). ~960 rows. Enables real-time anomaly detection.

hourly_by_dimension

Hourly rollup broken by **9 dimensions** via ARRAY JOIN + dictGet. ~69K rows. Powers segment-level detection without scanning raw events.

Aggregate Columns

```
requests_s → countState()
fills_s → sumState(is_filled)
imps_s → sumState(is_impression)
clicks_s → sumState(is_click)
revenue_s → sumState(revenue)
```

Dictionaries

`apps_dict` category, publisher_tier

`advertisers_dict` vertical, campaign_type

`geo_dict` region, country, device_model, os_version

DETECTION ENGINE

Five detectors run in parallel every cycle

Each detector compares the current window against a **same-day-of-week baseline** (3-week lookback, minimum 3 observations). An anomaly requires *both* statistical significance AND real-world magnitude.

Detector A · Robust Z-Score

Metrics: fill_rate, eCPM, CTR
Method: Median + IQR/1.35 for sigma
Fires when: |z| > 5 AND |Δ%| > 3%

Detector B · Trend Volume

Metrics: requests (raw count)
Method: Linear regression slope on baseline
Fires when: volume deviates from trend

Detector C · CUSUM

Metrics: fill_rate, eCPM
Method: Bidirectional cumulative sum
Fires when: sustained shift (K=0.5σ, H=4σ)

Detector D · Segment Z-Score (region)

Metrics: fill_rate, eCPM per region
Source: hourly_by_dimension rollup
Catches: segment anomalies invisible at platform level

Detector E · Segment Z-Score (os_version)

Metrics: fill_rate, eCPM per OS version
Source: hourly_by_dimension rollup
Catches: device-specific degradations

Why robust z-score? Standard mean/stddev is skewed by the very outliers we're trying to detect. Median + IQR is resistant to them.

TECH STACK

Lean, fast, ClickHouse-native

ClickHouse Cloud
Storage + Compute + All statistical math

ClickStack (HyperDX)
Managed observability — traces, logs, metrics

Go
API server, detection orchestration

Vanilla JS + React
Frontend dashboard, no build step

OpenTelemetry
Instrumentation → ClickStack

Oracle Cloud + ngrok
VM hosting + public tunnel

What makes it ClickHouse-native

- **AggregatingMergeTree** with -State/-Merge for incremental rollups
- **Materialized Views** as insert triggers — zero ETL
- **COMPLEX_KEY_HASHED dictionaries** for dimension joins
- **All stats in SQL:** quantile, IQR, median, linear regression — no Python, no external libs
- **ClickStack (HyperDX)** for end-to-end observability

Performance

- MV rollup query: **~5ms** (vs ~500ms on raw 4.5M rows)
- Full detection cycle (5 detectors): **~3s**
- Drilldown (11 parallel queries): **~5s**
- Total end-to-end: **under 10 seconds**

CH

Thank you

Live demo → math-mahogany-dish.ngrok-free.dev/dashboard

Source → github.com/clickathon-hope/clickathon-solution-2026

TEAM HOPE · CLICK-A-THON 2026